

SESSION 5 / 8 | 7 Mayıs 2026 | 90 dakika

React Best Practices

Component Parçalama, State Yönetimi, Custom Hooks



Session Akışı

Süre	Konu	Anahtar Kavramlar
0-25 dk	Component Parçalama	SRP, reusable component, prop drilling
25-45 dk	State Yönetimi Yaklaşımı	lifting state up, prop drilling
45-65 dk	Custom Hooks	use* prefix, mantık paylaşımı
65-90 dk	Pratik: ToDo Refactor	component split + custom hook

1 Component Parçalama

 0–25 dakika

1.1 Neden Component Parçalamalıyız?

Session 4’te yazdığımız ToDo uygulaması tek bir `ToDoApp` component’inde 100+ satır kod barındırıyordu. Bu yaklaşım küçük projelerde çalışır ama uygulama büyüdükçe sorunlar başlar:

- Kod okunması zorlaşır — her şey aynı dosyada
- Aynı UI parçası birkaç yerde lazım olunca kopyalanır
- Hata ayıklamak zorlaşır — nerede hata var?
- Test yazmak zor olur
- Birden fazla geliştirici aynı dosyada çalışınca çakışmalar artar

Single Responsibility Principle (SRP)

Her component **tek bir iş** yapmalıdır. Bir component “hem listeyi göster, hem de ekleme formunu yönet, hem de validation yap” demek yerine; **her sorumluluk ayrı bir component** olmalıdır.

1.2 Önce / Sonra Karşılaştırması

1.2.1 Önce — Tek Büyük Component

✗ TodoApp.tsx — Her şey tek dosyada

```
function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState("");

  // ... ekleme, silme, toggle fonksiyonlari ...

  return (
    <div>
      <h1>Todos</h1>

      { /* Form --- 20 satir kod */ }
      <form onSubmit={handleAdd}>
        <input value={text} onChange={...} />
        <button type="submit">Add</button>
      </form>

      { /* Liste --- 30 satir kod */ }
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <input type="checkbox" .../>
            <span>{todo.text}</span>
            <button onClick={...}>Delete</button>
          </li>
        ))}
      </ul>

      { /* Özet --- 10 satir kod */ }
      <p>{remaining} remaining</p>
    </div>
  );
}
```

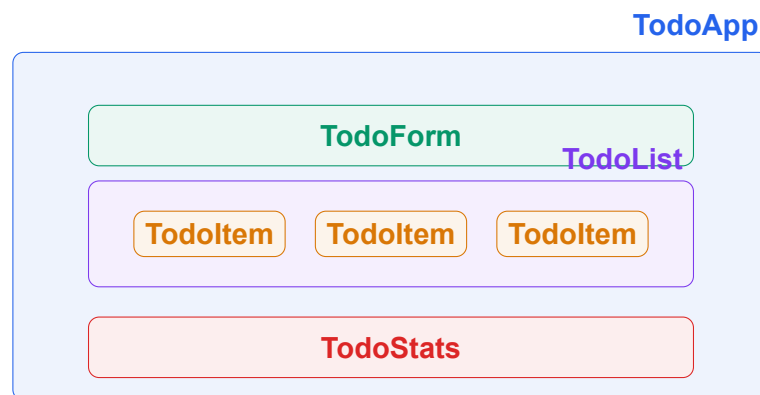
1.2.2 Sonra — Parçalanmış Component'ler

✓ TodoApp.tsx — Sadece koordinasyon

```
function TodoApp() {
  const [todos, setTodos] = useState([]);

  const addTodo = (text) => { /* ... */ };
  const toggleTodo = (id) => { /* ... */ };
  const deleteTodo = (id) => { /* ... */ };

  return (
    <div>
      <h1>Todos</h1>
      <TodoForm onAdd={addTodo} />
      <TodoList
        todos={todos}
        onToggle={toggleTodo}
        onDelete={deleteTodo}
      />
      <TodoStats todos={todos} />
    </div>
  );
}
```



1.3 Component Parçalama Sinyalleri

Bir component'i ne zaman parçalamalıyım? Aşağıdaki sinyalleri arayın:

Sinyal	Açıklama
Çok uzun JSX	100+ satır return bloğu varsa parçalama zamanı geldi.
Tekrarlayan yapı	Aynı JSX bloğu <code>map</code> dışında 2+ yerde varsa component yap.
Anlaşılması zor isim	"Bu <code><div></code> ne işe yarıyor?" diye düşünüyorsanız ona isim verin.
Bağımsız davranış	Bir bölüm kendi state/effect'ini yönetiyorsa ayrı component olabilir.
Yeniden kullanım	Aynı UI başka bir sayfada da lazım olacaksa component yapın.

💡 Sezgisel Kural

Bir component'i **üst üste 2–3 ekran kaydırarak** okumak zorunda kalıyorsanız, parçalama vakti gelmiş demektir. "Ne yapıyor?" sorusuna tek cümlede cevap veremiyorsanız da öyle.

1.4 Reusable (Yeniden Kullanılabilir) Component

Reusable component, farklı yerlerde **farklı veriyle** aynı görünümü sağlayabilen component'tir. Anahtar nokta: davranış props ile dışarıdan kontrol edilir.

1.4.1 Kötü — Sabit Veriye Bağımlı

✗ Yeniden kullanılamaz

```
function PrimaryButton() {
  return (
    <button
      onClick={() => alert("Saved!")}
      style={{ background: "#2563eb", color: "white" }}
    >
      Save
    </button>
  );
}
```

❗ Sorun

Bu buton sadece "Save" yazısı için, sadece alert için, sadece mavi renk için çalışır. Başka yerde kullanmak için kopyala-yapıştır yapmanız gerekir.

1.4.2 İyi — Props ile Esnek

✓ Reusable Button

```
interface ButtonProps {
  children: React.ReactNode;
  onClick?: () => void;
  variant?: "primary" | "danger" | "ghost";
  disabled?: boolean;
}

function Button({
  children,
  onClick,
  variant = "primary",
  disabled = false,
}: ButtonProps) {
  const colors = {
    primary: { bg: "#2563eb", fg: "white" },
    danger: { bg: "#ef4444", fg: "white" },
    ghost: { bg: "transparent", fg: "#2563eb" },
  };

  const c = colors[variant];

  return (
    <button
      onClick={onClick}
      disabled={disabled}
      style={{
        background: c.bg, color: c.fg,
        padding: "10px 20px", border: "none",
        borderRadius: 8, cursor: "pointer",
        opacity: disabled ? 0.5 : 1,
      }}
    >
      {children}
    </button>
  );
}

// Kullanım --- aynı component, farklı amaçlar
<Button onClick={save}>Save</Button>
<Button variant="danger" onClick={remove}>Delete</Button>
<Button variant="ghost" onClick={cancel}>Cancel</Button>
```

💡 Reusable Component Tarifi

1. İçeriği children prop'undan al
2. Davranışı (onClick, onChange...) prop'tan al
3. Görünümü variant, size gibi prop'larla yönet

4. Default değerleri tanımla (`variant = "primary"`)
5. Component **ne göstereceğini** bilmesin — sadece **nasıl görüneceğini** bilsin

2 State Yönetimi Yaklaşımı

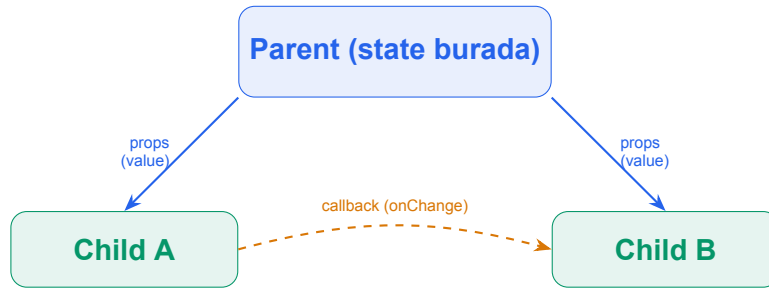
 25–45 dakika

2.1 State Nerede Yaşamalı?

Component'leri parçaladıkça yeni bir soru çıkar: **state'i nereye koyalım?**

Lifting State Up (Hatırlama)

Birden fazla component'in **aynı veriye** ihtiyacı varsa, o veri (state) onların **ortak atasında** (parent) tutulmalıdır. Aşağıya **props**, yukarıya **callback** ile akar.



2.1.1 Lifting State Up Örneği

İki sıcaklık inputu (Celsius ve Fahrenheit), ikisi de aynı sıcaklığı temsil ediyor. State'i ortak ata olan TemperatureCalculator'a taşırız:

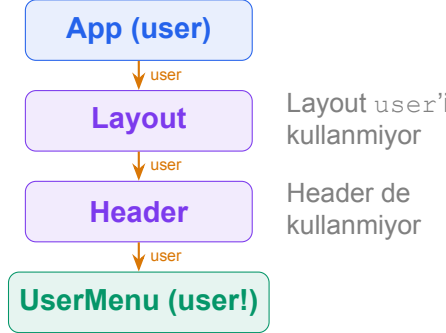
Lifting State Up

```
function TemperatureCalculator() {  
  // State, hem Celsius hem Fahrenheit input'unun ihtiyaci  
  // olan veridir.  
  // Bu yuzden ortak parent'a koyduk.  
  const [celsius, setCelsius] = useState(0);  
  const fahrenheit = celsius * 9 / 5 + 32;  
  
  return (  
    <div>  
      <CelsiusInput  
        value={celsius}  
        onChange={setCelsius}  
      />  
      <FahrenheitInput  
        value={fahrenheit}  
        onChange={(f) => setCelsius((f - 32) * 5 / 9)}  
      />  
    </div>  
  );  
}  
  
function CelsiusInput({ value, onChange }) {  
  return (  
    <label>  
      Celsius:  
      <input  
        type="number"  
        value={value}  
        onChange={(e) => onChange(Number(e.target.value))}  
      />  
    </label>  
  );  
}  
  
function FahrenheitInput({ value, onChange }) {  
  return (  
    <label>  
      Fahrenheit:  
      <input  
        type="number"  
        value={value}  
        onChange={(e) => onChange(Number(e.target.value))}  
      />  
    </label>  
  );  
}
```

2.2 Props Drilling — ve Sınırı

Props Drilling

State'i kullanmak için onu **birkaç katman aşağıdaki** component'e taşımak gerekiyorsa, aradaki tüm component'lere prop olarak geçirmek zorundayız. Buna **props drilling** denir.



⚠ Props Drilling Ne Zaman Sorun?

- Ara katmanlar veriyi **sadece geçirmek için** alıyorsa
- 3+ katman derinliğinde aynı prop dolaşıyorsa
- Yeni bir prop eklerken **birçok dosyayı düzenlemek** gerekiyorsa

💡 Çözüm Hiyerarşisi

1. **Önce:** 1–2 katman ise sorun değil, bırakın — en basit çözüm.
2. **Component composition:** `children` ile veriyi ara katmanlardan geçirmek mümkün.
3. **React Context:** Tema, dil, kullanıcı gibi global veriler için.
4. **State management kütüphaneleri:** Redux, Zustand, Jotai — karmaşık uygulamalar için (bu kursta detaya girmiyoruz).

2.3 State Yönetiminde Kararlar

Soru	Cevap
Tek component kullanıyor mu?	Lokal <code>useState</code>
İki kardeş component kullanıyor mu?	Ortak parent'a kaldır (lifting up)
Çok derinde sadece bir yerde lazım mı?	Component composition (<code>children</code>)
Birçok yerde global lazım mı?	React Context veya state lib
Sunucudan gelen veri mi?	Server state lib (TanStack Query — ile-ride)

Altın Kural

Önce **en yakın ortak parent**'ı seç. Çok kalabalık olursa parçala. Daha gelişmiş çözümleri sadece **ihtiyaç** duyduğunda getir.

3 Custom Hooks

 45–65 dakika

3.1 Custom Hook Nedir?

Custom Hook

Custom hook, **ismi use ile başlayan** ve içinde diğer hook'ları (`useState`, `useEffect`...) çağıran bir **JavaScript fonksiyonudur**. Component değildir, JSX döndürmez — **state mantığını** döndürür.

3.1.1 Neden Custom Hook?

Birden fazla component'te aynı state mantığını kullanıyorsanız — *örneğin `localStorage`'dan değer okuyup yazma, API'den veri çekip `loading/error` yönetme* — bu mantığı her component'te kopyala-yapıştır yapmak yerine bir custom hook'a çıkarırsınız.

Component	Custom Hook
JSX (UI) döndürür	Veri / fonksiyon döndürür
Büyük harfle başlar (<code><Card></code>)	<code>use</code> ile başlar (<code>useToggle</code>)
<code><Component /></code> ile çağrılır	Fonksiyon gibi çağrılır (<code>useToggle()</code>)
Görünüm paylaşır	Mantık paylaşır

3.2 İlk Custom Hook — `useToggle`

Açma/kapama (boolean) state'i React'ta sürekli kullanılır: modal aç/kapa, detay göster/gizle, tema değiştir... Her seferinde aynı kodu yazmak yerine:

3.2.1 Önce — Tekrarlayan Kod

✗ Aynı pattern her component'te

```
function Modal() {
  const [isOpen, setIsOpen] = useState(false);
  const open = () => setIsOpen(true);
  const close = () => setIsOpen(false);
  const toggle = () => setIsOpen(prev => !prev);
  // ... 50 satir UI ...
}

function Sidebar() {
  const [isOpen, setIsOpen] = useState(false);
  const open = () => setIsOpen(true);
  const close = () => setIsOpen(false);
  const toggle = () => setIsOpen(prev => !prev);
  // ... 50 satir UI ...
}
```

3.2.2 Sonra — Custom Hook

✓ useToggle.ts

```
import { useState } from "react";

export function useToggle(initial = false) {
  const [value, setValue] = useState(initial);

  const open = () => setValue(true);
  const close = () => setValue(false);
  const toggle = () => setValue(prev => !prev);

  return { value, open, close, toggle };
}
```

Kullanım — Tek Satırda

```
function Modal() {
  const { value: isOpen, toggle, close } = useToggle();

  return (
    <>
      <button onClick={toggle}>Toggle Modal</button>
      {isOpen && (
        <div className="modal">
          <button onClick={close}>X</button>
          <p>Modal content...</p>
        </div>
      )}
    </>
  );
}

function Sidebar() {
  const { value: isOpen, toggle } = useToggle(true);
  // ...
}
```

3.3 Pratik Custom Hook — useLocalStorage

Session 4 ToDo'da localStorage'dan okuma/yazma kodu useState ve useEffect'in birleşimiydi. Bunu tekrar tekrar yazmak yerine:

useLocalStorage.ts

```
import { useState, useEffect } from "react";

export function useLocalStorage<T>(key: string, initial: T) {
  // Lazy initial: localStorage'dan oku
  const [value, setValue] = useState<T>(() => {
    const stored = localStorage.getItem(key);
    return stored ? JSON.parse(stored) : initial;
  });

  // Deger degisince localStorage'a yaz
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
}
```

Kullanım — useState Gibi

```
function Settings() {
  // Tipki useState gibi kullanilir, ama sayfa yenilense bile
  // saklanir
  const [theme, setTheme] = useLocalStorage("theme", "light");
  const [lang, setLang] = useLocalStorage("lang", "tr");

  return (
    <div>
      <button onClick={() => setTheme(theme === "light" ? "
        dark" : "light")}>
        Toggle Theme: {theme}
      </button>
    </div>
  );
}
```

💡 Önemli Detay

useLocalStorage'ın dönüş değeri [value, setValue] formatında — aynı useState gibi. Bu sayede mevcut useState kodunu **tek satır değişiklikle** kalıcı hale getirebilirsiniz.

3.4 Custom Hook Kuralları

Kural	Açıklama
use prefix	İsim mutlaka <code>use</code> ile başlamalı (<code>useToggle</code> , <code>useFetch</code>). React linter buna göre çalışır.
Sadece tepe seviye	Diğer hook'lar gibi <code>if</code> , <code>for</code> , fonksiyon içinde çağrılmaz.
Component veya hook içinde	Custom hook'u sadece bir component veya başka bir hook içinde çağırabilirsiniz. Normal fonksiyonda çağrılmaz.
Değer döner	Ne döndürdüğünü siz seçersiniz: dizi (<code>[value, set]</code>), obje (<code>{ value, open, close }</code>), tek değer...
Tek sorumluluk	Bir custom hook tek bir iş yapmalı. <code>useUserAndCart</code> yerine <code>useUser + useCart</code> .

3.4.1 Yaygın Custom Hook Örnekleri

- `useToggle` — aç/kapa state
- `useLocalStorage` — `localStorage` senkronizasyonu
- `useFetch` — API çağrısı + `loading/error` yönetimi
- `useDebounce` — arama input'ları için `debounce`
- `useWindowSize` — pencere boyutu takibi
- `usePrevious` — bir değer `önceki` değerini hatırla
- `useClickOutside` — modal/dropdown dışına tıklama

🔗 Bonus — useFetch İskeleti

```
function useFetch<T>(url: string) {  
  const [data, setData] = useState<T | null>(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState<string | null>(null);  
  
  useEffect(() => {  
    setLoading(true);  
    fetch(url)  
      .then(r => r.json())  
      .then(d => { setData(d); setLoading(false); })  
      .catch(e => { setError(e.message); setLoading(false);  
        });  
  }, [url]);  
  
  return { data, loading, error };  
}  
  
// Kullanım  
const { data, loading, error } = useFetch<User[]>("/api/  
  users");
```


4 Pratik: ToDo Uygulamasını Refactor Etme

 65–90 dakika

☰ Pratik Hedef

Session 4'te yazdığımız tek dosyalık ToDo uygulamasını bu session'da öğrendiğimiz prensiplere göre refactor edeceğiz:

1. Component'leri parçala: `TodoForm`, `TodoList`, `TodoItem`, `TodoStats`
2. State'i `TodoApp`'te tut, props ile aşağıya akıt
3. `localStorage` mantığını `useLocalStorage` hook'una çıkar
4. Reusable `Button` component'i kullan

4.1 Hedef Dosya Yapısı

```
src/  
├─ App.tsx  
├─ components/  
│   ├─ Button.tsx  
│   ├─ TodoForm.tsx  
│   ├─ TodoList.tsx  
│   ├─ TodoItem.tsx  
│   └─ TodoStats.tsx  
└─ hooks/  
    └─ useLocalStorage.ts
```

4.2 Adım 1 — useLocalStorage Hook'u

src/hooks/useLocalStorage.ts

```
import { useState, useEffect } from "react";  
  
export function useLocalStorage<T>(key: string, initial: T) {  
  const [value, setValue] = useState<T>(() => {  
    const stored = localStorage.getItem(key);  
    return stored ? JSON.parse(stored) : initial;  
  });  
  
  useEffect(() => {  
    localStorage.setItem(key, JSON.stringify(value));  
  }, [key, value]);  
  
  return [value, setValue] as const;  
}
```

4.3 Adım 2 — TodoForm Component'i

src/components/TodoForm.tsx

```
import { useState } from "react";

interface Props {
  onAdd: (text: string) => void;
}

export function TodoForm({ onAdd }: Props) {
  const [text, setText] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!text.trim()) {
      setError("Task cannot be empty!");
      return;
    }
    onAdd(text.trim());
    setText("");
    setError("");
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Add a task..."
      />
      <button type="submit">Add</button>
      {error && <p style={{ color: "red" }}>{error}</p>}
    </form>
  );
}
```

💡 Dikkat

TodoForm, todos dizisini **görmüyor** bile. Sadece “yeni metin geldi” diye onAdd callback’ini çağırıyor. Bu, **Single Responsibility Principle**’in göstergesidir.

4.4 Adım 3 — TodoItem ve TodoList

src/components/TodoItem.tsx

```
interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

interface Props {
  todo: Todo;
  onToggle: (id: number) => void;
  onDelete: (id: number) => void;
}

export function TodoItem({ todo, onToggle, onDelete }: Props)
{
  return (
    <li style={{
      textDecoration: todo.completed ? "line-through" : "none"
    ,
      color: todo.completed ? "#94a3b8" : "#1e293b",
    }}>
      <input
        type="checkbox"
        checked={todo.completed}
        onChange={() => onToggle(todo.id)}
      />
      <span>{todo.text}</span>
      <button onClick={() => onDelete(todo.id)}>X</button>
    </li>
  );
}
```

src/components/ToDoList.tsx

```
import { TodoItem } from "../TodoItem";

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

interface Props {
  todos: Todo[];
  onToggle: (id: number) => void;
  onDelete: (id: number) => void;
}

export function ToDoList({ todos, onToggle, onDelete }: Props)
{
  if (todos.length === 0) {
    return <p>No tasks yet. Add one above!</p>;
  }

  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem
          key={todo.id}
          todo={todo}
          onToggle={onToggle}
          onDelete={onDelete}
        />
      ))}
    </ul>
  );
}
```

4.5 Adım 4 — TodoStats

src/components/TodoStats.tsx

```
interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

interface Props {
  todos: Todo[];
}

export function TodoStats({ todos }: Props) {
  if (todos.length === 0) return null;

  const remaining = todos.filter(t => !t.completed).length;
  const completed = todos.filter(t => t.completed).length;

  return (
    <p>
      {remaining} remaining
      {completed > 0 && ` - ${completed} completed`}
    </p>
  );
}
```

4.6 Adım 5 — App.tsx (Koordinator)

src/App.tsx — Sade ve net

```
import { useLocalStorage } from "../hooks/useLocalStorage";
import { TodoForm } from "../components/TodoForm";
import { TodoList } from "../components/TodoList";
import { TodoStats } from "../components/TodoStats";

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

function App() {
  const [todos, setTodos] = useLocalStorage<Todo[]>("todos", []);

  const addTodo = (text: string) => {
    setTodos([...todos, { id: Date.now(), text, completed: false }]);
  };

  const toggleTodo = (id: number) => {
    setTodos(todos.map(t =>
      t.id === id ? { ...t, completed: !t.completed } : t
    ));
  };

  const deleteTodo = (id: number) => {
    setTodos(todos.filter(t => t.id !== id));
  };

  return (
    <div>
      <h1>Todos</h1>
      <TodoForm onAdd={addTodo} />
      <TodoList
        todos={todos}
        onToggle={toggleTodo}
        onDelete={deleteTodo}
      />
      <TodoStats todos={todos} />
    </div>
  );
}

export default App;
```

💡 Refactor Kazanımları

- `App.tsx` sadece **40 satır** ve ne yaptığı bir bakışta anlaşılıyor
- Her component'in **tek sorumluluğu** var
- `TodoForm` ve `TodoItem` başka projelerde de kullanılabilir
- `useLocalStorage` hook'u her tür state için yeniden kullanılabilir
- Test yazmak çok daha kolay — her parça bağımsız test edilebilir

5 Özet ve Sonraki Session

5.1 Bu Session'da Öğrendiklerimiz

- ✓ Single Responsibility Principle (SRP)
- ✓ Component parçalama sinyalleri
- ✓ Reusable component yazma (props ile esneklik)
- ✓ State'in nerede yaşaması gerektiği
- ✓ Lifting state up
- ✓ Props drilling kavramı ve sınırı
- ✓ Custom hook nedir, ne değildir
- ✓ `useToggle`, `useLocalStorage` örnekleri
- ✓ Custom hook kuralları (`use` prefix, tepe seviye...)
- ✓ Pratik: ToDo uygulamasını refactor etme

5.2 Sıkça Sorulan Sorular

SSS

S: Her şeyi component'e mi parçalamalıyım?

C: Hayır. Aşırı parçalama (over-engineering) da sorun yaratır. “Şu an okuması zor mu? Tekrar mı ediyorum? Yeniden kullanacak mıyım?” sorularına “evet” cevabı varsa parçala. Yoksa olduğu gibi bırak.

S: Custom hook ile component arasındaki fark nedir?

C: Component **görünüm** (JSX) döndürür. Custom hook **mantık** (state, fonksiyon, hesaplanmış değer) döndürür. İkisi de hook'ları kullanabilir ama amaçları farklıdır.

S: Props drilling 2–3 katman varsa hemen Context kullanmalı mıyım?

C: Hayır. Önce **component composition** (`children`) ile çözmeyi dene. Context'i sadece veri **gerçekten global** ise (tema, kullanıcı, dil) kullan.

S: Custom hook ne zaman yazmalıyım?

C: Aynı state mantığını **2+ component**'te kullanıyorsanız çıkarın. “Bunu da hatırlamak istemiyorum” dediğiniz noktada custom hook zamanı gelmiştir.

5.3 Sonraki Session — Session 6: Next.js Giriş

→ Session 6 (11 Mayıs Pazartesi)

React bilginizi Next.js ile genişletmeye başlıyoruz:

- **Next.js neden var** — React'a göre farkları
- **SSR vs CSR** — basit anlatım
- **File-based routing** ve `Link` kullanımı
- **Layout** yapısı ve sayfa organizasyonu
- **Pratik**: Liste ve detay sayfası oluşturma